Computer Networks Lab (CSE/PC/B/S/314): Assignment 4 Report

CDMA Multiple Access Using Walsh Codes

1. Title

Field	Details
Name	Soumyadip Modak
Class	(BCSE- III /3rd Year / 1st Semester)
Group	A!
Assignment number	4
Course	CSE/PC/B/S/314 Computer Networks Lab
Code demonstration deadline	05-09 October 2026
Report submission deadline	
Date of submission	
Language / platform	C, POSIX UDP sockets (Linux, Ubuntu on VirtualBox)

Problem statement. Implement Code Division Multiple Access (CDMA) for n stations that share one common channel. Every Sender is given a unique code word from the Walsh set and uses it to spread its data bits. The senders transmit at the same time, the Channel adds all the signals together, and the Receiver uses the Walsh codes to separate the combined signal back into the data of each of the n stations. The same sender-receiver design as in the previous assignments is kept: a Sender reads a test file, builds 46-byte-payload frames, appends the checksum and CRC from Assignment 1, and sends them with stop-and-wait; the Receiver acknowledges each frame.

2. Design

2.1 Purpose of the program

The program simulates a shared wireless-style medium with three cooperating programs. The **Sender** (Sender.c, with stop_wait.h) is run once per station. It splits the file into 46-byte datawords, builds a frame, spreads every bit of the frame with its own Walsh code and sends the resulting chips over UDP to the Channel (port 4455). The **Channel** (Channel.c) behaves as the physical medium: in every time slot it collects one transmission from each active station, adds the chip streams sample by sample and forwards the single composite signal to the Receiver. The **Receiver** (Receiver.c, port 4456) correlates the composite signal with the Walsh code of each station, rebuilds each station's frame, checks the sequence number, prints the recovered data and returns an ACK.

Shared headers: walsh.h (Walsh table generation), checksum.h and Crc.h (from Assignment 1). The files go_back_n.h and selective_repeat.h are carried over from Assignment 2 but are not used here; Sender.c includes only stop_wait.h.

2.2 Frame structure

The frame layout is the same as in Assignment 1: header + payload + checksum + CRC. The header holds source MAC (6), destination MAC (6), length (2) and sequence number (2) bytes. With CRC-8 the frame is 65 bytes, i.e. 520 bits.

Field	Size (bytes)	Notes
Header	16	Src MAC, Dest MAC (zero), length (network order), sequence number (0 or 1)
Payload	46	Next 46 bytes of the file; last chunk zero-padded, true length stored in the header
Checksum	2	16-bit one's complement checksum over header + payload (62 bytes), little-endian
CRC-8	1	Polynomial 0xD5 over the first 62 bytes
Total	65	65 × 8 = 520 bits per frame

The ACK frame is 2 bytes (ack_no). The end of a station's transmission is signalled by a 4-byte datagram containing the text DONE. Both ACK and DONE are small control messages and are **not** spread.

Spreading. Each frame bit becomes N chips, so a frame of L bytes produces L × 8 × N chips, each stored as a signed 16-bit integer. For the 65-byte frame: N = 2 gives 1,040 chips, N = 4 gives 2,080 and N = 8 gives 4,160.

2.3 Structure diagram (procedural organisation)

```
SENDER (Sender.c)  -- one process per station
main()
|-- read n (total senders) and this station ID
|-- walsh_N = next_power_of_2(n); generateWalshTable();
walsh_code[] = row[ID]
|-- parse argv (receiver IP, test file), open file, create UDP
socket, choose CRC
`-- send_stop_and_wait()
    |-- fread 46 bytes (zero-pad last chunk)
    |-- build frame: [16B header][46B payload]
    |-- cal_chksum() -> append 2 bytes (checksum.h)
    |-- cal_crc8()   -> append 1 byte  (Crc.h)
    |-- csma_sendto(): bit -> +1/-1, multiply by walsh_code[],
send chips to Channel
    |-- wait up to 400 ms for 2-byte ACK; on timeout retransmit
    `-- send "DONE"


CHANNEL (Channel.c)
main(Receiver_IP, n)
|-- bind port 4455; walsh_N = next_power_of_2(n); active_senders
= n
`-- loop (one iteration = one CDMA slot):
    |-- select() with 2 s timeout, receive datagrams until one
per active sender
    |-- from port 4456 (ACK) -> relay to every registered sender
    |-- "DONE"              -> active_senders--, forward DONE to
Receiver
    |-- chips               -> composite[i] += chips[i]
    `-- sendto(Receiver, composite)


RECEIVER (Receiver.c)
main()
|-- read n and CRC choice; generateWalshTable(); bind port 4456
`-- loop: recvfrom()
    |-- "DONE" -> count; stop when all n stations finished
    |-- frame_bytes = num_chips / (8 * N)
    |-- for each station st: correlate composite with
walsh_table[st] -> bits -> frame
    |-- valid length and expected seq -> accept and print data;
else discard duplicate
    `-- send 2-byte ACK to Channel
```

2.4 Input and output format

Sender input: ./sender <Receiver_IP> <Test_File>, then interactive prompts for the total number of senders n, this station's ID (0 to n−1) and the CRC width. **Sender output:** a line for each frame stating the number of chips transmitted, the sequence number sent, the ACK received (or a timeout message), and a final completion message.

Channel input: ./channel <Receiver_IP> <Num_Senders>. **Channel output:** the Walsh dimension N, then one line per slot with the number of senders superimposed and chips forwarded, and a line for every DONE signal.

Receiver input: ./receiver, then n and the CRC choice. **Receiver output:** for each slot and each station a line [Station k] Seq s Accepted | Data: "..." (or DISCARDED (Duplicate)), and the number of stations that have sent DONE.

Test data: Test.txt, 477 bytes, giving 11 frames per station (10 full frames plus one final chunk of 17 bytes zero-padded to 46).

3. Implementation

3.1 Walsh codes (walsh.h)

next_power_of_2(n) returns the smallest power of two that is at least n (and at least 2), which gives the code length N. generateWalshTable(N, table) builds the N × N matrix with Sylvester's construction: the top-left, top-right and bottom-left quadrants copy the previous matrix and the bottom-right quadrant is its negation. For n = 3 the matrix is 4 × 4 and station i uses row i:

Station	Code	Note
0	W0 = [+1, +1, +1, +1]	Used
1	W1 = [+1, −1, +1, −1]	Used
2	W2 = [+1, +1, −1, −1]	Used

Any two different rows have inner product 0 and every row has inner product N with itself. This is what lets the Receiver remove the other stations completely.

3.2 Spreading at the Sender (csma_sendto in Sender.c)

The original csma_sendto() from the previous assignment is replaced by a function with the same signature so that stop_wait.h needs no change. For every bit of the

frame (MSB first) the Sender maps 1 to +1 and 0 to −1, multiplies it by each of the N chips of its Walsh code and stores the results as int16_t. Messages of 5 bytes or fewer (the DONE signal) are sent unchanged.

```
for (size_t i = 0; i < len; i++)
  for (int bit = 7; bit >= 0; bit--) {
    int bipolar = ((frame[i] >> bit) & 1) ? 1 : -1;
    for (int c = 0; c < walsh_N; c++)
      chips[chip_idx++] = bipolar * walsh_code[c];
  }
```

3.3 Superposition in the Channel (Channel.c)

The Channel keeps an array composite[] that is cleared at the start of every slot. It then receives datagrams until it has one from each active sender. A datagram from port 4456 is an ACK from the Receiver and is copied to all registered senders; the first datagram from a new address registers that sender; a short datagram starting with DONE reduces the number of active senders and is forwarded to the Receiver; any other datagram is a chip stream and is added element by element. The composite vector is then sent to the Receiver. A 2-second select() timeout prevents a deadlock if a datagram is lost: a partly filled slot is forwarded as it is.

```
int16_t *chips = (int16_t *)buffer;
int num_chips = bytes / sizeof(int16_t);
for (int i = 0; i < num_chips && i < MAX_CHIPS; i++)
    composite[i] += chips[i];          /* linear
superposition */
```

3.4 Correlator at the Receiver (Receiver.c)

From the length of the composite signal the Receiver computes frame_bytes = num_chips / (8 × N). For each station st it takes N chips at a time, multiplies them by row st of the Walsh table, sums, and divides by N. A positive result is bit 1 and anything else is bit 0. Eight bits form a byte and the bytes form the frame.

```
for (int c = 0; c < walsh_N; c++)
    sum += composite[chip_idx++] * walsh_table[st][c];
int bipolar = sum / walsh_N;           /* = d_st, the station's
bit */
if (bipolar > 0) reconstructed[b] |= (1 << bit);
```

A station that is silent contributes 0 to every chip, so its correlation is 0, all its bits decode as 0 and the header length field is 0. The Receiver only accepts a frame when the length is between 1 and 46, so silent or finished stations are skipped without any special signalling.

3.5 Reliability: stop-and-wait with a slot ACK

Each Sender sends one frame, then waits up to 400 ms (in 50 ms steps) for a 2-byte ACK whose number equals its sequence number, and retransmits on timeout. The sequence number alternates between 0 and 1. The Receiver keeps expected_seq[st] for each station. If the received sequence number matches, the frame is accepted and the expected value toggles; otherwise the frame is printed as a duplicate and discarded. One ACK is returned per slot to the Channel, which relays it to all registered senders.

3.6 Worked example (n = 3, N = 4)

Suppose in one bit slot station 0 sends 1, station 1 sends 0 and station 2 sends 1, so d = (+1, −1, +1). The Channel adds the three chip vectors:

Quantity	Value	Decoded bit
S = W0 − W1 + W2	[+1, +3, −1, +1]	–
(S · W0) / 4	(1 + 3 − 1 + 1) / 4 = +1	1
(S · W1) / 4	(1 − 3 − 1 − 1) / 4 = −1	0
(S · W2) / 4	(1 + 3 + 1 − 1) / 4 = +1	1
(S · W3) / 4	(1 − 3 + 1 + 1) / 4 = 0	silent

4. Test Cases

Every test uses Test.txt (477 bytes, 11 frames per station) and CRC-8. The Channel, Receiver and all Senders must be started with the same n.

#	Test	Input / setting	What is checked
T1	Non-power-of-2 stations	n = 3, stations 0, 1, 2	N rounds up to 4; 2,080 chips per slot; all three stations decode; W3 unused
T2	Power-of-2 stations	n = 2	N = 2; 1,040 chips per slot; two stations decode
T3	Full code set	n = 4	N = 4; all four Walsh rows in use at once
T4	Larger spreading factor	n = 5	N = 8; 4,160 chips per slot; rows W5–W7 unused

#	Test	Input / setting	What is checked
T5	Different data per station	Different file for each Sender	No crosstalk: each station's output matches its own file
T6	Unequal file lengths	One long file, one short file	After the short sender sends DONE it is silent; the other station is unaffected
T7	Last-chunk padding	File length not a multiple of 46	Final 17-byte chunk is padded; the length field limits the printed data
T8	Termination	End of all files	Channel and Receiver count n DONE signals and stop
T9	Missing sender	Start n−1 senders for n expected	Channel slot timeout (2 s) forwards a partial composite instead of hanging
T10	Wrong n	Receiver started with a different n than the Senders	Decoding fails, showing that n and N must be identical at all three programs

5. Results

5.1 Spreading overhead (computed from the frame size)

For the 65-byte frame (520 bits) and 16-bit chips, the size of a spread frame depends only on N:

N	Max stations	Chips per frame	UDP payload (bytes)	Within Channel limit?
2	2	1,040	2,080	Yes
4	4	2,080	4,160	Yes
8	8	4,160	8,320	Yes
16	16	8,320	16,640	Yes
32	32	16,640	33,280	Yes
64	64	33,280	66,560	No (> 32,000 chips, > 65,507 bytes)

A frame bit becomes N chips of 16 bits each, so the data on the wire grows by a factor of 16N in this simulation (64× for N = 4).

5.2 Observed run: three stations (T1)

Figure 1 shows the run for n = 3. The top row has the three Sender terminals (stations 0, 1 and 2, CRC-8, Test.txt). The bottom left terminal is the Receiver (./receiver, n = 3) and the bottom right terminal is the Channel (./channel 10.0.2.15 3).

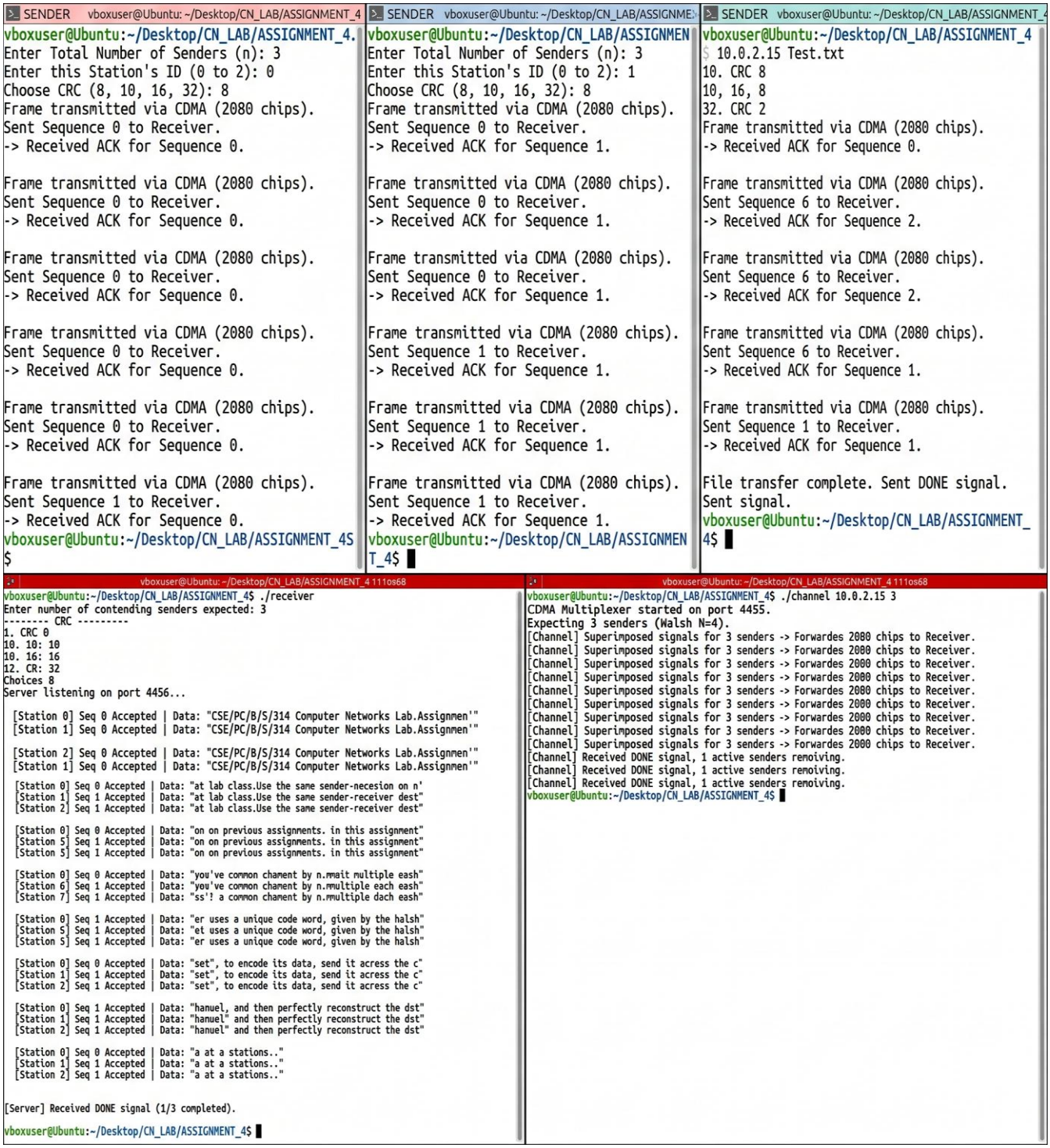


Figure 1: Output of the CDMA run with n = 3 (N = 4): three Senders, the Channel and the Receiver.

What the output shows:

- The Channel reports that it expects 3 senders and uses Walsh N = 4, as n = 3 is rounded up to the next power of 2.
- Each Sender reports that a frame was transmitted via CDMA as 2,080 chips, which is 520 bits × 4, matching Section 2.2.
- For each slot the Channel prints that it superimposed the signals of 3 senders and forwarded the chips to the Receiver. Only one composite stream reaches the Receiver, even though three stations transmitted together.
- The Receiver separates the composite stream and prints lines for Station 0, Station 1 and Station 2, each with the text recovered from Test.txt.
- When the Senders finish, the Channel logs the DONE signals and the Receiver counts them and exits.

5.3 Summary of test outcomes

#	n	N	Chips / frame	Expected result	Observed
T1	3	4	2,080	3 stations decode, W3 silent	As expected (Figure 1)
T2	2	2	1,040	2 stations decode	(fill in after running)
T3	4	4	2,080	4 stations decode	(fill in after running)
T4	5	8	4,160	5 stations decode, W5–W7 silent	(fill in after running)
T5–T10	–	–	–	See Section 4	(fill in after running)

6. Analysis

6.1 Reading the results

The results follow from the orthogonality of the Walsh rows. When the Channel adds the chips of all stations, the Receiver's inner product with W_k gives N times the bit of station k and exactly 0 for every other station, so there is no interference between stations and no collision, unlike CSMA/CD in Assignment 3 where simultaneous transmissions have to be detected, jammed and retried after a random backoff. Here all stations transmit in the same slot and nothing is lost.

Silent stations are handled by the same maths. A station that has finished (or a Walsh row that was never assigned, such as W3 for n = 3) adds nothing to the composite, so correlating with its code gives 0. The Receiver sees a zero length field and ignores it.

The largest chip value is n in magnitude (all stations sending the same bit), which is at most 64 for this code, so the 16-bit chip buffers cannot overflow.

6.2 Why N must be a power of 2

Sylvester's construction only produces matrices of size 1, 2, 4, 8 and so on. For n stations, N is therefore 2 raised to the ceiling of log2(n), and N − n codes are left unused. With n = 5 this costs N = 8, so 3 of the 8 codes are unused.

6.3 Known issues of this implementation

- **The Receiver does not verify the checksum or CRC.** Both are computed and appended by the Sender, but Receiver.c only checks the length field and the sequence number. The CRC choice entered at the Receiver is read but not used. Error injection in Sender.c is also stubbed out (injkterror is empty), so no error experiments are run in this assignment.
- **Only CRC-8 is appended.** stop_wait.h builds the CRC only when crc_choice == 8. For 10, 16 or 32 the frame is sent with the checksum alone (64 bytes). The Receiver still works because it derives the frame size from the chip count.
- **One shared ACK per slot.** The Receiver sends a single ACK per slot, based on the first station with a valid frame, and the Channel relays it to every Sender. This works when all stations stay in step, but if one station is retransmitting while another has moved on, a Sender can receive an ACK number that does not match its own sequence number and will retransmit.
- **Slots are formed by arrival count, not by station.** The Channel closes a slot after receiving one datagram per active sender, without checking which sender it came from. If a station retransmits while another is waiting, two chip streams from the same station can land in one slot and the other station is missed in that slot.
- **Size limits.** MAX_CHIPS is 32,000, the Channel buffer is 65,000 bytes and a UDP datagram holds at most 65,507 bytes, so with 520-bit frames the largest usable N is 32. MAX_WALSH_SIZE is 64, which cannot be reached. The 4,160-byte datagram for N = 4 also exceeds the 1,500-byte Ethernet MTU, so on a real network it would be fragmented.

- **Output only on screen.** The Receiver prints the recovered text but does not write per-station output files, so diff or hash comparison with the input is not possible yet.
- **Same n everywhere.** The value of n is entered separately in each program. If it differs, the Walsh dimension differs and decoding fails (T10).

6.4 Trade-offs

CDMA removes collisions and backoff delay, and all stations transmit in the same time slot. The price is bandwidth: every bit is replaced by N chips, which needs N times the bandwidth on a real channel and requires strict chip synchronisation between the stations. The Channel here provides that synchronisation by slotting. With Walsh codes the stations must be aligned exactly; with unaligned stations the cross-correlation is no longer 0 and Gold or other pseudo-random codes would be needed.

6.5 Possible improvements

1. Verify the checksum and the CRC at the Receiver, enable the error injector, and implement CRC-10, CRC-16 and CRC-32 in stop_wait.h.
2. Send a per-station ACK (station ID plus sequence number) so that each Sender is acknowledged independently.
3. Tag every chip datagram with the station ID so that the Channel builds slots per station.
4. Write each station's recovered data to output_station_k.txt and compare it with the input using diff and md5sum.
5. Split large composites into several datagrams, or use a lower-overhead representation of chips (for example 8-bit values), to remove the size limit and fragmentation.
6. Add noise to the channel and measure the bit error rate for different N.

7. Comments

This assignment showed how several stations can use the same channel at the same time without collisions when each one has an orthogonal code. Writing the Walsh generator and the correlator made the idea of inner products and orthogonality concrete, and the worked example with three stations helped to verify the Receiver by hand. Reusing the Sender and the stop-and-wait module from earlier assignments by only replacing csma_sendto() was convenient. The assignment was of moderate

difficulty: the coding is short, but making the Channel, the Receiver and the Senders agree on n, N and the slot structure needs care. A suggestion is to ask for per-station acknowledgments and file output so that the result can be verified automatically.

Appendix: source files

Sender.c (Sender and CDMA spreading), Channel.c (superposition channel), Receiver.c (correlator and ACK), walsh.h (Walsh table), stop_wait.h (stop-and-wait Sender), checksum.h (16-bit checksum), Crc.h (CRC-8/10/16/32), Test.txt (test input, 477 bytes). Present in the folder but not used in this assignment: go_back_n.h, selective_repeat.h.